Guía de desarrollo — Diego

Diego Alejandro Cely Ruiz · M1 Autenticación y roles · M4 Cotizaciones · Offero, versión del 27 de octubre de 2026

> Esta guía es tu hoja de ruta para tus módulos: qué construir, en qué orden, con qué tablas y permisos, y cómo saber que quedó bien. No trae código: el código lo escribes tú. Las historias con puntos van en el Product Backlog del plan (Parte 5). Si algo aquí choca con el documento de contexto, manda el documento y le avisas a Diego. Lo marcado con [SUPUESTO] es una propuesta que todavía hay que confirmar.

Tu resumen

Módulos	M1 Autenticación, registro y roles (complejidad alta) y M4 Cotizaciones (alta). ≈64 h al ritmo del equipo; con tu experiencia, ≈38 h [SUPUESTO: ≈60 % del tiempo de un principiante].
Otros roles	Product Owner, Scrum Master, dueño de las cuentas (GitHub, Vercel, Supabase), aprobación final de todo lo que toca base de datos y aplicación del SQL en producción.
Carga estimada	≈38 h de módulos + ≈13 h de revisiones ≈ 51 h frente a ≈47 h netas: estás un poco sobrecargado. Válvula: si te atrasas en M4, Iker toma «Mis cotizaciones».
Desbloqueas a	Todo el equipo (M1 y cuentas); Wilson (M5 necesita quotes) e Iker (M6 necesita cotizaciones aceptadas).
Dependes de	Daniel (M0; businesses) y Wilson (requests).
Fechas clave	Cuentas y proyectos sáb 31 oct · Vercel dom 1 nov · **M1 núcleo sáb 7 nov** · M1 terminado dom 15 nov · enviar cotización dom 22 nov · M4 terminado dom 29 nov.

> **Tu orden de prioridades:** 1) desbloquear a otros; 2) revisar pull requests en menos de 24 h; 3) tu propio código. Si los tres compiten, gana el primero.

Plataforma: tareas de dueño de las cuentas

Tarea	Cuándo	Detalle
Etiqueta `prototipo-v0` sobre main	mié 28 oct	Conserva el prototipo antes de que Daniel limpie el repositorio.
Colaboradores en App-Offero	mié 28 oct	Iker (fuentesiker), Wilson (wilsonher65-hash) y Daniel (arven1203; en el documento aparece Arven1203-art: confirma el usuario exacto al invitarlo).
Proteger main	mié 28 oct	Exigir pull request con 1 aprobación; bloquear force push y borrado.
Supabase: offero-dev y offero-prod	sáb 31 oct	[SUPUESTO: el plan gratuito permite dos proyectos activos; verifícalo.] Invita al equipo a la organización con el rol mínimo necesario, si el plan lo permite.
Auth en ambos proyectos	sáb 31 oct	URL del sitio y URLs de redirección para localhost, previews de Vercel y producción. Sin esto, las redirecciones fallan.
Decisión sobre correos de confirmación	sáb 31 oct	El correo incluido en Supabase solo llega a miembros del equipo y tiene un límite muy bajo. Opciones: desactivar la confirmación en el MVP, o SMTP propio (agrega un proveedor externo → decisión pendiente por la restricción 1). Sin SMTP tampoco funcionará «recuperar contraseña» para usuarios reales. Registrarla en la sección 11.
Vercel: importar App-Offero desde tu cuenta	dom 1 nov	Con Daniel. Variables: Production → offero-prod; Preview → offero-dev.

Tarea	Cuándo	Detalle
Fusionar con «Create a merge commit»	Siempre	Así quedas como autor y Vercel Hobby no bloquea el despliegue.
Aplicar SQL en offero-prod	Al fusionar cada PR con SQL	El mismo archivo que se probó en dev; anota en el PR que ya se aplicó.
Pasar claves	Cuando M0 esté en main	URL y clave pública de offero-dev por mensaje privado. La clave secreta (service_role) no se comparte ni va al frontend.
Revisar offero-prod antes de cada demo	Antes de demos	[Verificar] Los proyectos gratuitos de Supabase pueden pausarse por inactividad.

Tu calendario por sprint

Sprint	Qué entregas
0 · 28 oct – 8 nov	Etiqueta, colaboradores y protección de main; offero-dev y offero-prod; Vercel con Daniel; decisión de correos; M1 núcleo (sáb 7 nov); acordar con el profesor el compromiso por niveles.
1 · 9 – 15 nov	Sesión del contrato de autenticación (lun 9); M1 terminado (dom 15); primeras entrevistas con empresas (PO).
2 · 16 – 22 nov	Tabla quotes + RLS; QuoteForm (dom 22); revisar las tablas de Daniel y Wilson; en la Review: decidir si entran los extras de M7.
3 · 23 – 29 nov	Comparación y aceptación; M4 terminado (dom 29); demostración con usuarios reales.
4 · 30 nov – 6 dic	Revisión de seguridad de todas las tablas con Iker; apoyo a M5, M6 y M7; preparar la presentación.
5 · 7 – 11 dic	Estabilización y presentación al profesor.

Lo que te toca enseñar (sesiones compartidas)

Ya dominas el stack. Tu aprendizaje es de facilitación: que lo que sabes deje de depender de ti.

Sesiones

1. Sprint 0: Git en equipo y flujo de pull requests (sobre todo para Wilson e Iker).
2. Lun 9 nov: el contrato de autenticación: obtener el usuario y su perfil en el servidor, proteger una página y una Server Action, usar `is_admin()` y `get_my_role()`.
3. Sprint 1: RLS en la práctica: escribir una política y probarla con dos cuentas.
4. Sprint 2: cómo leer los errores de Supabase y los logs de compilación de Vercel.

Para repasar tú

1. `@supabase/ssr` con la versión de Next.js que quede instalada (en Next.js 16 el archivo de middleware pasó a llamarse proxy).
2. Funciones con privilegios elevados en PostgreSQL: validar quién llama y fijar el search_path.

Tus roles de PO y SM sin ser cuello de botella

- **PO:** mantener el backlog ordenado; aceptar o rechazar historias según sus criterios; acordar con el profesor esta semana el compromiso por niveles; liderar las entrevistas con las empresas de Bogotá; definir las categorías iniciales.
- **SM:** no facilites todas las ceremonias. La Daily asíncrona y la Retrospectiva las facilita alguien distinto cada sprint (Iker, Daniel, Wilson). Como PO eres parte interesada en la Retro; un facilitador neutral ayuda a que salgan los problemas reales.

- **Revisiones:** respondes cada PR en menos de 24 h. Los PR que no tocan base de datos, autenticación ni archivos compartidos los puede aprobar otro compañero.
- **Factor bus:** todo lo que solo tú sabes (cuentas, SQL de producción, contrato de autenticación) queda escrito en el README o en `docs/`.

Módulo 1 — Autenticación, registro y roles

Objetivo	Registro, inicio y cierre de sesión con Supabase Auth; perfil con rol; protección de rutas y operaciones según el rol; y un «contrato» de autenticación documentado que todo el equipo reutiliza.
Alcance	Mínimo = completo. Todo M1 es obligatorio y bloquea a todos los demás módulos.
Fuera de alcance	Inicio con Google (aunque ya lo dominas, no entra salvo que sobre tiempo); recuperación de contraseña, que depende de la decisión sobre los correos; verificación de identidad de empresas.
Depende de	Daniel (M0) para las páginas. La parte de base de datos la puedes adelantar en offero-dev desde el primer día.
Desbloquea a	A todos.

Tabla `profiles`

Campo	Tipo sugerido	Reglas
id	uuid = auth.users.id	Clave primaria; se borra en cascada si se borra el usuario de Auth.
full_name	text	Obligatorio, 2 a 100 caracteres.
email	text	Copia informativa. Nunca se muestra a otros usuarios.
role	text	client, provider o admin (CHECK); por defecto client.
created_at	timestamptz	Automático.
is_suspended (opcional)	boolean	Solo si Iker hace la suspensión de cuentas en M7 (extra). Acuérdenlo antes de crear la tabla.

Creación automática del perfil: una función y un trigger que se ejecutan al registrarse un usuario en Auth. Toman nombre y rol de los datos del registro y solo aceptan client o provider: si llega admin u otro valor, guardan client. Así nadie se registra como admin manipulando la petición.

Vista `public_profiles`: solo id y full_name, para mostrar nombres en cotizaciones y mensajes sin exponer correos.

Funciones de ayuda (entregable para todos): `is_admin()` y `get_my_role()`. No uses el nombre current_role: es una palabra reservada de PostgreSQL. Documenta en el README cuándo usar cada una.

Políticas RLS de `profiles`

Operación	Quién	Condición
SELECT	Usuario	Solo su propio perfil; el admin ve todos.
INSERT	Nadie desde la app	Lo hace el trigger.
UPDATE	Usuario	Solo su full_name. Nadie cambia su propio rol; el cambio de rol por el admin lo agrega Iker en M7.
DELETE	Nadie en el MVP	—

Rutas y archivos

- `src/lib/supabase/` — cliente de servidor (con cookies) y cliente de navegador, con `@supabase/ssr`. Nadie más crea clientes.
- Archivo de sesión en `src/` — en Next.js 16 se llama `proxy.js` (antes `middleware.js`); confírmalo con la versión instalada. Refresca la sesión en cada petición.

- `src/services/auth.js` — obtener el usuario y su perfil en el servidor; exigir un rol (redirige si no cumple).
- `src/app/registro/page.js` — nombre, correo, contraseña y rol (cliente o proveedor).
- `src/app/login/page.js` — correo y contraseña.
- `src/app/perfil/page.js` — ver datos y rol; editar el nombre.
- Cerrar sesión desde el encabezado (coordina con Daniel, dueño del layout).
- README, sección «Autenticación»: el contrato explicado en texto: cómo obtener el usuario, cómo proteger una página y una Server Action, y cómo usar las funciones de ayuda en una política.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Cuentas y entornos (ver «Plataforma»).	sáb 31 oct	offero-dev y offero-prod existen y Auth está configurado en ambos.
2	SQL de profiles, trigger, vista public_profiles, funciones de ayuda y RLS en offero-dev.	mar 3 nov	Registrar un usuario desde el panel crea su perfil con rol client.
3	Clientes de Supabase y archivo de sesión.	mié 4 nov	La sesión se mantiene al recargar.
4	Registro, inicio y cierre de sesión con Server Actions.	vie 6 nov	Dos cuentas de prueba en dev (un cliente y un proveedor) funcionan.
5	Patrón de protección por rol (página y Server Action) y README. **Hito «M1 núcleo»: el equipo ya puede trabajar.**	sáb 7 nov	Sin sesión, /perfil redirige a /login.
6	Sesión de enseñanza del contrato (45 min, con todos).	lun 9 nov	Cada uno sabe proteger una página y usar `get_my_role()` en una política.
7	Página de perfil (ver y editar nombre).	mié 11 nov	El nombre cambia; el rol no se puede cambiar.
8	Tu cuenta como admin en producción (rol asignado por SQL: es un dato real).	jue 12 nov	Puedes entrar a /administrador cuando Iker lo cree.
9	Prueba cruzada (la hace Iker: intentar volverse admin y entrar a rutas ajenas) y publicación.	dom 15 nov	Checklist de la Definition of Done completo (Anexo G).

Criterios de aceptación

- **Dado** un visitante, **cuando** se registra como proveedor con datos válidos, **entonces** se crea su cuenta y su perfil con rol provider.
- **Dado** alguien que manipula la petición de registro para enviar el rol admin, **cuando** se crea la cuenta, **entonces** su rol queda como client.
- **Dado** un cliente con sesión, **cuando** intenta cambiar su propio rol desde la app o directo contra Supabase, **entonces** la operación falla.
- **Dado** un usuario sin sesión, **cuando** abre /perfil, /solicitudes/nueva o /administrador, **entonces** es enviado a /login.
- **Dado** un usuario que cerró sesión, **cuando** usa el botón Atrás del navegador, **entonces** no ve información protegida.

Errores típicos y cómo evitarlos

- En el servidor, tomar decisiones de seguridad con la sesión de la cookie sin validarla contra Supabase Auth.
- El trigger falla y el registro devuelve un error genérico de base de datos: revisa los logs de Postgres en el panel.
- URLs de redirección sin configurar para las previews de Vercel: los enlaces fallan solo en las previews.
- Páginas con datos del usuario que quedan en caché: deben ser dinámicas.
- Una clave secreta dentro de una variable `NEXT_PUBLIC_`.

Coordinación con el equipo

- Todos: el sáb 7 nov anuncia en Discord que el «M1 núcleo» está listo y dónde está la documentación.
- Daniel: acuerden cómo va el menú de sesión (entrar, salir, Mi negocio, Panel) en el encabezado.
- Iker: él agrega la política de admin para cambiar roles y revisa tu trigger y tus funciones como parte de seguridad.

Módulo 4 — Cotizaciones y comparación de propuestas

Objetivo	Los proveedores con negocio envían cotizaciones a solicitudes abiertas; el cliente las compara por precio, plazo y condiciones y elige una; el sistema actualiza todos los estados a la vez.
Alcance	Mínimo = completo. Todo M4 es obligatorio (Must) y cierra el flujo principal del MVP.
Fuera de alcance	Contraofertas o negociación, adjuntar archivos a la cotización, pagos (excluidos por el documento) y avisos por correo.
Depende de	M1 (tuyo); businesses (Daniel, jue 12 nov); requests (Wilson, mié 11 nov).
Desbloquea a	Wilson (M5: quién puede escribir a quién) e Iker (M6: quién puede reseñar).

Tabla `quotes`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
request_id	uuid → requests.id	Obligatorio; se borra en cascada.
business_id	uuid → businesses.id	Obligatorio.
price	numeric(12,2)	Mayor que 0, en pesos colombianos.
delivery_time	integer	Días de entrega, mayor que 0 [SUPUESTO: el documento no fija la unidad; en días se puede ordenar].
conditions	text	Opcional, hasta 1000 caracteres.
valid_until	date	Igual o posterior a la fecha de envío.
status	text	pending, accepted, rejected o withdrawn; por defecto pending.
created_at	timestamptz	Automático.
Única (request_id, business_id)	—	Un negocio cotiza una sola vez por solicitud.

Políticas RLS de `quotes`

Operación	Quién	Condición
SELECT	Proveedor, cliente, admin	El dueño del negocio que cotizó; el cliente dueño de la solicitud; el admin.
INSERT	Proveedor	Es dueño de `business_id`; la solicitud está open y no oculta; status pending.
UPDATE	Proveedor	Solo retirar su cotización (pending → withdrawn). El cliente no actualiza directamente: acepta mediante la función de la base de datos.
DELETE	Nadie	Se retira en vez de borrar.

> **Aceptar una cotización es una operación atómica.** Es una función de la base de datos que, en una sola transacción, verifica que quien llama es el dueño de la solicitud, que la solicitud está open y que la cotización está pending y vigente. Luego deja esa cotización accepted, las demás rejected y la solicitud awarded. Si se hiciera con tres llamadas desde Next.js y una fallara, quedarían datos inconsistentes. Como la función actúa con privilegios elevados, debe validar ella misma quién llama; Iker la revisa como parte de seguridad.

Rutas y archivos

- `src/components/quotes/QuoteForm.js` — formulario de cotización; Wilson lo coloca para los proveedores en el detalle de la solicitud (`src/app/solicitudes/[requestId]/page.js`).
- `src/app/cotizaciones/[requestId]/page.js` — comparación para el cliente: negocio, precio, plazo, vigencia y condiciones; ordenar por precio o plazo; botón Elegir solo si la solicitud está open; las vencidas se marcan.
- `src/app/cotizaciones/page.js` [SUPUESTO: ruta nueva] — «Mis cotizaciones» del proveedor, con estado y opción de retirar.
- `src/components/quotes/` — QuoteComparisonTable y QuoteStatusBadge.
- `src/services/quotes.js` — consultas y llamada a la función de aceptar.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Tabla quotes + RLS en dev. Avisa a Wilson e Iker.	mar 17 nov	El proveedor A no ve la cotización de B; el cliente ve ambas.
2	QuoteForm con validación en el servidor; si el proveedor no tiene negocio, enlace a «Mi negocio».	dom 22 nov	Una cotización válida queda pending; una segunda del mismo negocio se rechaza.
3	«Mis cotizaciones» del proveedor y retirar.	mar 24 nov	Retirar cambia el estado y la cotización sale de la comparación.
4	Página de comparación con orden por precio y por plazo.	mié 25 nov	Con 3 cotizaciones, el orden por precio es correcto.
5	Función de aceptar + botón Elegir + estados visibles. Avisa a Iker.	vie 27 nov	Al elegir, los tres estados cambian a la vez.
6	Prueba cruzada (la hace Daniel) y demostración en producción con un cliente y dos proveedores reales.	dom 29 nov	Flujo solicitud → cotización → elegir funcionando en Vercel.

Criterios de aceptación

- **Dado** un proveedor con negocio y una solicitud abierta, **cuando** envía precio, plazo, condiciones y vigencia válidos, **entonces** la cotización queda pendiente y el cliente la ve en la comparación.
- **Dado** un proveedor que ya cotizó una solicitud, **cuando** intenta enviar otra cotización a la misma, **entonces** el sistema lo impide.
- **Dado** un cliente con tres cotizaciones, **cuando** ordena por precio, **entonces** ve primero la más barata.
- **Dado** un cliente, **cuando** elige una cotización vigente, **entonces** esa queda aceptada, las demás rechazadas y la solicitud adjudicada, todo a la vez.
- **Dado** el proveedor B, **cuando** intenta ver las cotizaciones de otros proveedores en la misma solicitud, **entonces** no puede.
- **Dado** un cliente, **cuando** llama la función de aceptar sobre una solicitud ajena, **entonces** falla.

Errores típicos y cómo evitarlos

- Guardar o comparar precios como texto: el orden sale mal.
- Fechas de vigencia con hora y zona horaria: usa solo fecha (Bogotá es UTC−5).
- Doble clic en Elegir: deshabilita el botón y haz que la función falle si la solicitud ya no está open.
- Mostrar el correo del cliente al proveedor: usa la vista public_profiles.

Coordinación con el equipo

- Wilson: acuerden dónde va QuoteForm en el detalle de la solicitud y el ajuste del SELECT de requests para que los proveedores sigan viendo las solicitudes que cotizaron.
- Daniel: necesitas su función «negocio del usuario actual».
- Wilson (M5) e Iker (M6): avísales cuando quotes exista en dev (mar 17 nov) y cuando la función de aceptar esté lista (vie 27 nov).
- Si vas atrasado, Iker puede tomar la vista «Mis cotizaciones» del proveedor (tiene margen en el Sprint 3).

Anexo — Acuerdos comunes del equipo

Este anexo es igual en las cuatro guías. Si cambia, se cambia en todas.

A. Calendario del equipo

Sprint	Fechas	Meta del equipo
0	mié 28 oct – dom 8 nov	Proyecto publicado (M0), entornos listos, todos con su computador preparado, M1 en marcha
1	lun 9 – dom 15 nov	M1 terminado; tablas categories, businesses, requests y reports creadas con RLS
2	lun 16 – dom 22 nov	M2 y M3 terminados; envío de cotizaciones; panel de admin básico
3	lun 23 – dom 29 nov	M4 terminado: solicitud → cotización → elegir, demostrable con usuarios reales
4	lun 30 nov – dom 6 dic	M5 y M6 (mínimos) terminados; M7 mínimo terminado
5	lun 7 – vie 11 dic	Estabilización: pruebas, errores y presentación. Entrega: vie 11 dic

Festivos en Colombia dentro del periodo: lun 2 nov, lun 16 nov y mar 8 dic.

B. Preparar tu computador (Windows, PowerShell)

#	Paso	Comando o acción	Cómo verificar
1	Instalar Node.js LTS	`winget install OpenJS.NodeJS.LTS` (o el instalador LTS de nodejs.org)	`node -v` y `npm -v` muestran versión
2	Instalar Git	`winget install Git.Git`	`git --version`
3	Permitir scripts de npm	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (responde S)	`npx -v` funciona sin el error «scripts deshabilitados»

4	Configurar Git	`git config --global user.name "Tu Nombre"`
		`git config --global user.email "correo de tu GitHub"`
		`git config --global init.defaultBranch main`
			`git config --global --list`

5	Extensiones de VS Code	ESLint, Tailwind CSS IntelliSense y Prettier; opcional: GitLens	Aparecen en la pestaña Extensiones

6	Clonar fuera de OneDrive	`mkdir C:\dev`
		`cd C:\dev`
		`git clone https://github.com/diegodani2030/App-Offero.git`
			Existe `C:\dev\App-Offero`

7	Instalar dependencias (cuando M0 esté en main)	`cd C:\dev\App-Offero` `npm install`	Termina sin errores en rojo
8	Variables de entorno	`Copy-Item .env.example .env.local` y llena los valores que Diego te envía por mensaje privado	`.env.local` existe y `git status` no lo muestra
9	Correr la app	`npm run dev` y abre http://localhost:3000	Ves la portada de Offero

Abre una PowerShell nueva después de instalar Node y Git. El correo de Git debe ser el de tu cuenta de GitHub. La primera vez que hagas push, Windows abrirá el navegador para iniciar sesión en GitHub.

C. Flujo de Git paso a paso

#	Qué haces	Comando o acción
1	Toma una tarea del tablero y muévela a «En curso»	GitHub Projects
2	Actualiza main	`git switch main` → `git pull`
3	Crea tu rama: tipo/mN-descripcion (tipos: feat, fix, chore, docs)	`git switch -c feat/m3-formulario-solicitud`
4	Guarda avances pequeños con mensajes en español: tipo(módulo): qué hace	`git add .` → `git commit -m "feat(m3): agrega formulario de nueva solicitud"`
5	Sube la rama	`git push -u origin feat/m3-formulario-solicitud` (después, solo `git push`)
6	Abre el pull request hacia main con la plantilla: qué cambia, cómo probarlo, capturas y checklist. Escribe «Cierra #N» para enlazar la tarea	GitHub
7	Revisión: al menos un compañero. Si toca base de datos, autenticación o archivos compartidos, revisa Diego. Corrige con nuevos commits en la misma rama	GitHub
8	Fusión: la hace Diego con «Create a merge commit» (por la restricción de Vercel Hobby); luego se borra la rama	GitHub
9	Actualiza tu rama con main cada día	`git switch main` → `git pull` → `git switch tu-rama` → `git merge main`
10	Si hay conflicto, VS Code muestra las opciones (cambio actual, entrante o ambos). Si no entiendes el otro cambio, pregúntale a su autor. Luego guarda y haz commit	`git add .` → `git commit`

- Nunca hagas commits directos a main ni `git push --force`.
- Pull requests de menos de un día de trabajo: son más fáciles de revisar y casi no generan conflictos.
- No subas `.env.local`, `node_modules` ni claves. Si `git status` los muestra, detente y pregunta.

D. Convenciones de base de datos

- **Dos proyectos de Supabase** [SUPUESTO; se confirma en la Parte 3]: `offero-dev` para desarrollo y pruebas (ahí sí puedes crear datos de prueba) y `offero-prod` solo con datos reales (lo usa la versión de producción en Vercel). Tu `.env.local` y las previews apuntan a dev.
- **Cada cambio de esquema es un archivo** en `supabase/sql/` con nombre `AAAAMMDD_HHMM_descripcion.sql` (por ejemplo `20261111_1530_requests.sql`). La fecha y hora evitan choques entre cuatro personas. Se prueba en dev, va en el PR y Diego lo aplica en prod al fusionar. Nunca edites un archivo ya aplicado: crea uno nuevo. Nadie cambia tablas desde el panel sin dejar su archivo.
- **Todas las tablas**: id uuid automático (profiles usa el id de Auth), created_at automático, claves foráneas explícitas y restricciones CHECK para estados y números.
- **RLS se activa en el mismo archivo que crea la tabla.** Una tabla sin políticas no se fusiona.
- **Contenido moderable** (businesses, requests, reviews) incluye `is_hidden` desde su creación, para que M7 no tenga que modificar tablas ajenas [SUPUESTO; registrar en la sección 11 del documento].
- **Funciones de ayuda de Diego**: `is_admin()` y `get_my_role()`. Úsenlas en las políticas en vez de repetir consultas.

- **Nombres de personas**: siempre desde la vista `public_profiles`. Los correos nunca se muestran a otros usuarios.
- **Validación en tres capas**: formulario (comodidad), servidor de Next.js (seguridad) y base de datos (restricciones y RLS, la última barrera).

Tabla.campo	Valores acordados (fuente única)
`profiles.role`	client, provider, admin
`requests.status`	open (recibe cotizaciones), awarded (adjudicada), cancelled
`quotes.status`	pending, accepted, rejected, withdrawn
`reports.status`	open, reviewed, dismissed
`reports.target_type`	business, request, review

E. Archivos compartidos

Archivo o carpeta	Dueño	Regla
`src/app/layout.js`, `globals.css`, `src/components/layout/`	Daniel	Avisa en Discord antes de cambiarlos; cambios pequeños
`src/lib/supabase/` y archivo de sesión (proxy o middleware)	Diego	Nadie crea otro cliente de Supabase
`supabase/sql/`	Dueño de cada tabla; aprueba Diego	No editar archivos ya aplicados
`package.json` y `package-lock.json`	Todos, con acuerdo	No instales paquetes sin acordarlo (restricción de stack). Si hay conflicto en el lock: acepta la versión de main y vuelve a ejecutar `npm install`
`docs/` (documento de contexto)	Cada uno, su módulo	Actualiza las secciones 8, 9 y 10 al avanzar
`src/components/ui/`	Quien crea el componente	Si cambias un componente compartido (ReportButton, CategoryFilter), avisa

F. Comunicación, regla de atasco e IA

- **Canales** [SUPUESTO]: Discord para el trabajo (#daily para el avance diario, #ayuda para bloqueos, #prs para pedir revisión). WhatsApp solo para urgencias.
- **Regla de atasco**: después de 45 minutos intentando solo, escribe en #ayuda qué intentabas, el archivo, el mensaje de error completo y qué probaste. Si en 2 horas nadie responde, menciona a Diego. Un bloqueo de más de un día se dice en la Daily.
- **Revisiones**: responde las que te pidan en menos de 24 horas.
- **IA**: pega el documento de contexto al iniciar cada conversación; pide explicaciones, no solo código; verifica que use JavaScript, App Router y `@supabase/ssr` (no TypeScript, ni la carpeta `pages/`, ni Flask); no subas nada que no puedas explicar en la revisión; nunca pegues claves en la IA.

G. Definition of Done

Una historia está hecha cuando:

- cumple sus criterios de aceptación;
- su pull request fue revisado y fusionado;
- funciona en la Preview y luego en producción;
- sus permisos se probaron con dos cuentas de roles distintos;

- no deja errores en la consola del navegador;
- el documento de contexto refleja cualquier cambio.

Un módulo está Terminado cuando (regla del documento de contexto):

- funciona en la versión publicada en Vercel;
- sus operaciones principales están probadas por otra persona (prueba cruzada);
- no depende de datos ficticios en producción;
- la sección 8 del documento dice Terminado y Publicado: Sí.